Dino, Grounding-Dino, SAM, DepthAnything

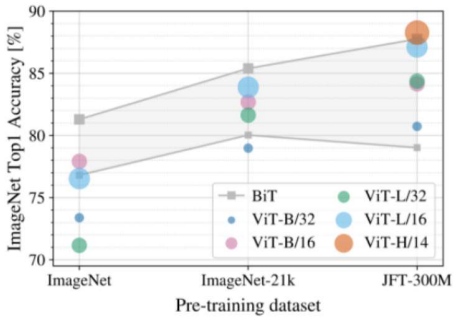
Vision transformer (ViT)

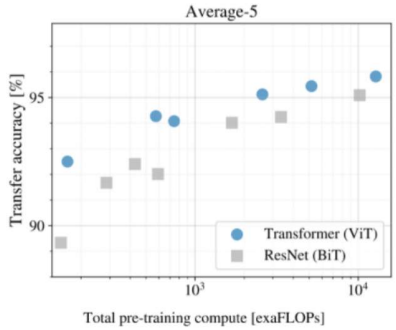
- The transformer architecture can also be applied to computer vision, by splitting images into a **sequence** of small patches (16x16).
- The sequence of patches can then be classified using the first output of the Transformer encoder (BERT) using supervised learning on Imagenet.

Source: https://ai.googleblog.com/2020/12/transformers-for-image-recognition-at.html

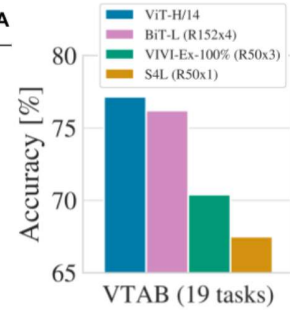
Vision transformer (ViT)

- The Vision Transformer (ViT) outperforms state-of-the-art CNNs on Imagenet while requiring less computations (Flops), but only when pretrained on bigger datasets.
- The performance is acceptable when trained on ImageNet (1M images), great when pre-trained on ImageNet-21k (14M images), and state-of-the-art when pre-trained on Google's internal JFT-300M dataset (300M images).
- Transfer learning on smaller datasets is also SotA.





	ViT-H	Previous SOTA
ImageNet	88.55	88.5
ImageNet-ReaL	90.72	90.55
Cifar-10	99.50	99.37
Cifar-100	94.55	93.51
Pets	97.56	96.62
Flowers	99.68	99.63

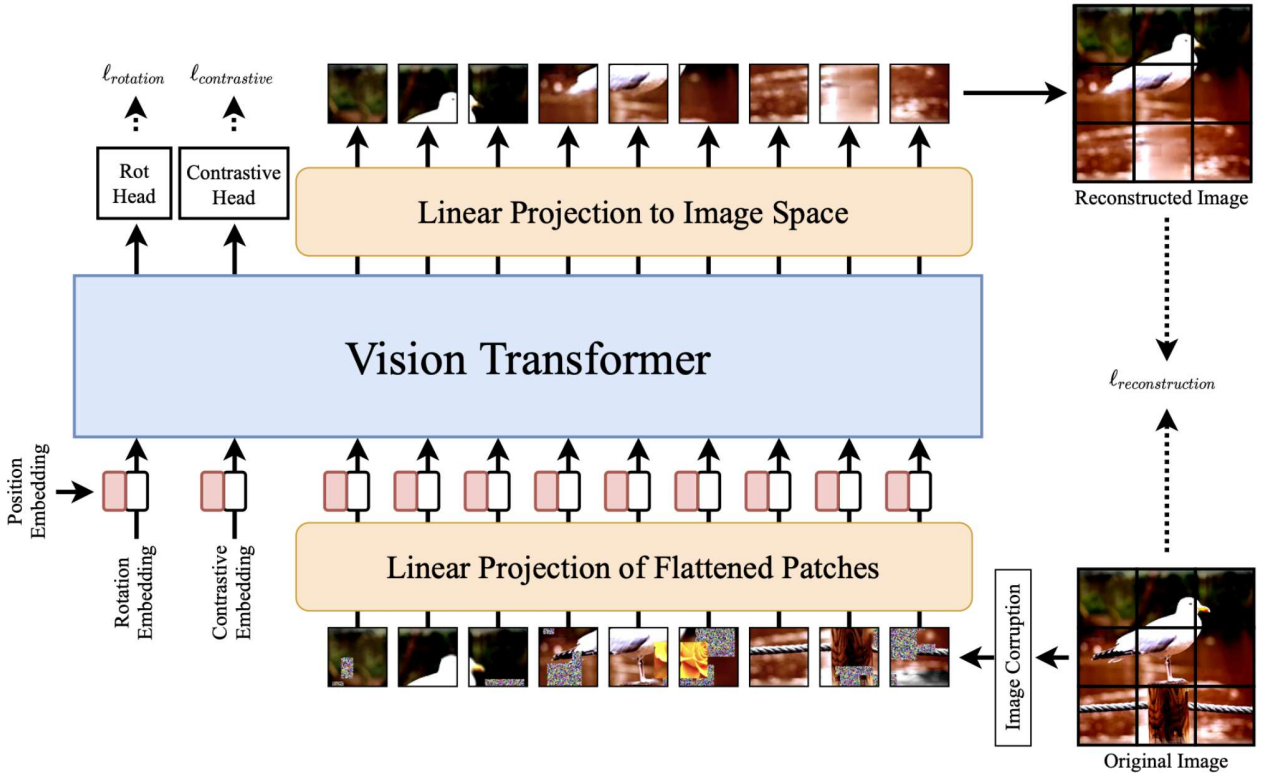


https://ai.googleblog.com/2020/12/transformers-for-image-recognition-at.html

2 - Self-supervised Vision Transformer

Self-supervised Vision Transformer (SiT)

- ViT only works on big supervised datasets (ImageNet). Can we benefit from self-supervised learning as in BERT or GPT?
- The Self-supervised Vision Transformer (SiT) has an denoising autoencoder-like structure, reconstructing corrupted patches autoregressively.



Self-supervised Vision Transformer (SiT)

- Self-supervised learning is possible through from **data augmentation**, where various corruptions (masking, replacing, color distortion, blurring) are applied to the input image, but SiT must reconstruct the original image (denoising autoencoder, **reconstruction loss**).



- An auxiliary **rotation loss** forces SiT to predict the orientation of the image (e.g. 30°).
- An auxiliary **contrastive loss** ensures that high-level representations are different for different images.

Method	Backbone	Linear Evaluation			Domain Transfer	
		CIFAR10	CIFAR100	Tiny-ImageNet	C100→C10	C10 →C100
DeepCluster [19]	ResNet-32	43.31% ± 0.62	20.44% ± 0.80	11.64% ± 0.21	43.39% ± 1.84	18.37% ± 0.41
RotationNet [23]	ResNet-32	62.00% ± 0.79	29.02% ± 0.18	14.73% ± 0.48	52.22% ± 0.70	27.02% ± 0.20
Deep InfoMax [20]	ResNet-32	47.13% ± 0.45	24.07% ± 0.05	17.51% ± 0.15	45.05% ± 0.24	23.73% ± 0.04
SimCLR [8]	ResNet-32	77.02% ± 0.64	42.13% ± 0.35	25.79% ± 0.4	65.59% ± 0.76	36.21% ± 0.16
SimCLR [8]	ResNet-56	78.75% ± 0.24	44.33% ± 0.48	n/a	66.19% ± 0.80	36.79% ± 0.45
Relational Reasoning [21]	ResNet-32	74.99% ± 0.07	46.17% ± 0.16	30.54% ± 0.42	67.81% ± 0.42	41.50% ± 0.35
Relational Reasoning [21]	ResNet-56	77.51% ± 0.00	47.90% ± 0.27	n/a	68.66% ± 0.21	42.19% ± 0.28
SiT (ours) - Linear projection	Transformer	81.98% ± 0.24	54.31% ± 0.13	40.35% ± 0.27	73.79% ± 0.15	55.72% ± 0.13
SiT (ours) - Non-Linear projection	Transformer	**83.50%** ± 0.11	**57.75%** ± 0.21	**43.06%** ± 0.14	**75.52%** ± 0.11	**57.89%** ± 0.14

Self-distillation with no labels (DINO)

- Another approach for self-supervised learning has been proposed by Facebook AI using **self-distillation**.
- The images are split into **global** and **local patches** at different scales.
- Global patches contain label-related information (whole objects) while local patches contain finer details.

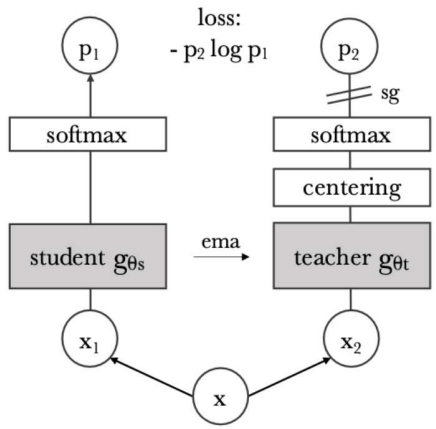


Davide Coccomini | 2021

Source: https://towardsdatascience.com/on-dino-self-distillation-with-no-labels-c29e9365e382

Self-distillation with no labels (DINO)

- The idea of **self-distillation** in DINO is to use two similar ViT networks to classify the patches.
- The **teacher** network gets the global views as an input, while the **student** network get both the local and global ones.
- Both have a MLP head to predict the softmax probabilities, but do **not** use any labels.

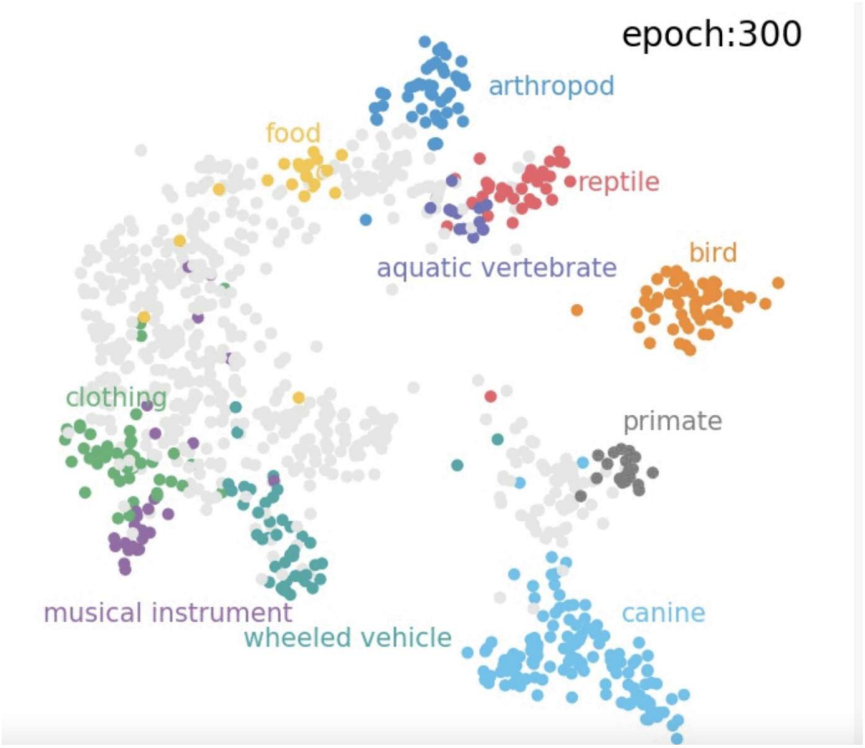


- The student tries to imitate the output of the teacher, by minimizing the **cross-entropy** (or KL divergence) between the two probability distributions.
- The teacher slowly integrates the weights of the student (momentum or exponentially moving average ema):

$$\theta_{\text{teacher}} \leftarrow \beta\,\theta_{\text{teacher}} + (1-\beta)\,\theta_{\text{student}}$$

Self-distillation with no labels (DINO)

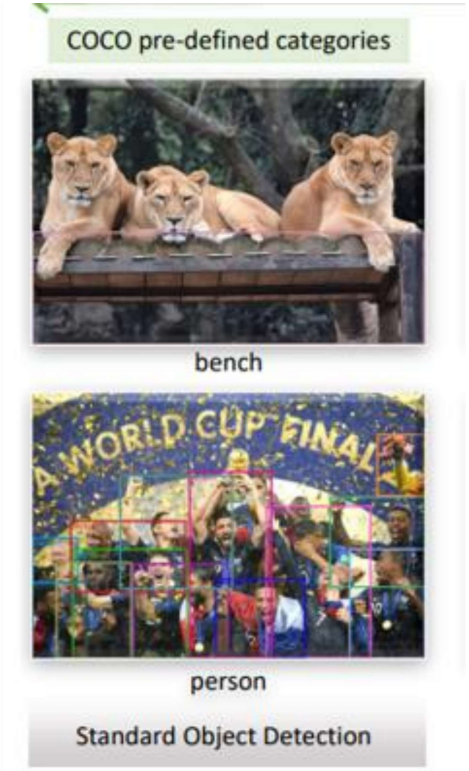
- The predicted classes do not matter when pre-training, as there is no ground truth.
- The only thing that matters is the **high-level representation** of an image before the softmax output, which can be used for transfer learning.
- Self-distillation forces the representations to be meaningful at both the global and local scales, as the teacher gets global views.
- ImageNet classes are already separated in the high-level representations: a simple kNN (k-nearest neighbour) classifier achieves 74.5% accuracy (vs. 79.3% for a supervised ResNet50).



https://ai.facebook.com/blog/dino-paws-computer-vision-with-self-supervised-transformers-and-10x-more-efficient-training

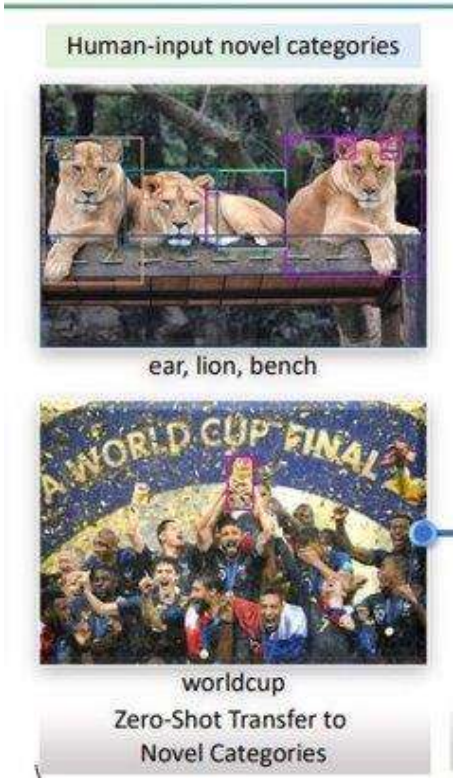
Closed-set Detection

Detect objects of pre-defined classes



Open-set Detection

Detect objects that were not present in training data



Grounding DINO

Introduction

- Strong Open-set object detector which combines transformer-based detector DINO with grounded pre-training
- Introduces language for unseen object generalization.
- Detects arbitrary objects with human inputs (Ex: category or referring expressions)

Grounding DINO

Closed-set Detection + Language → Open-set Detection

Transformer Based Detector (DINO) + Grounded pre training → Grounding DINO

➡ DINO - DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection

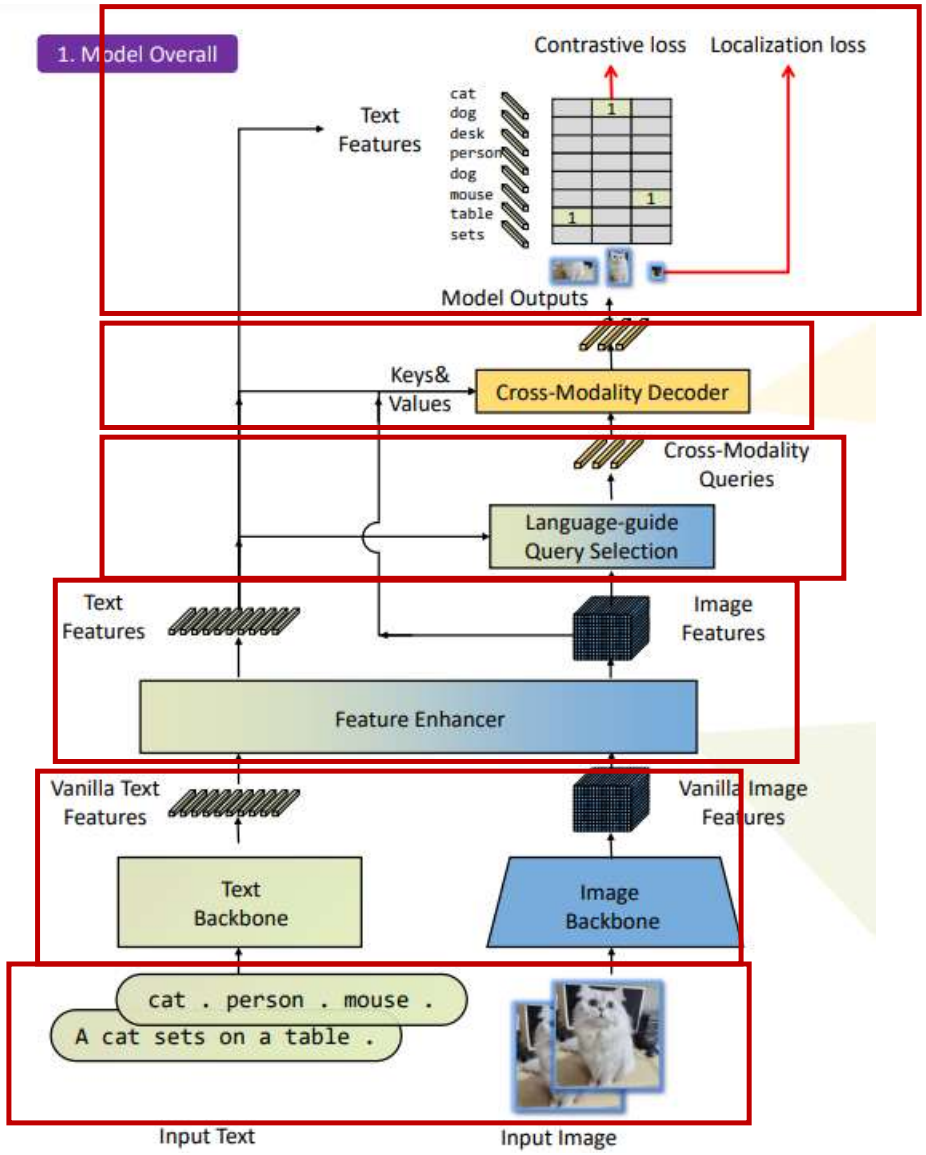
✖ DINO - Emerging Properties in Self-Supervised Vision Transformers

Related Work – Detection Transformers

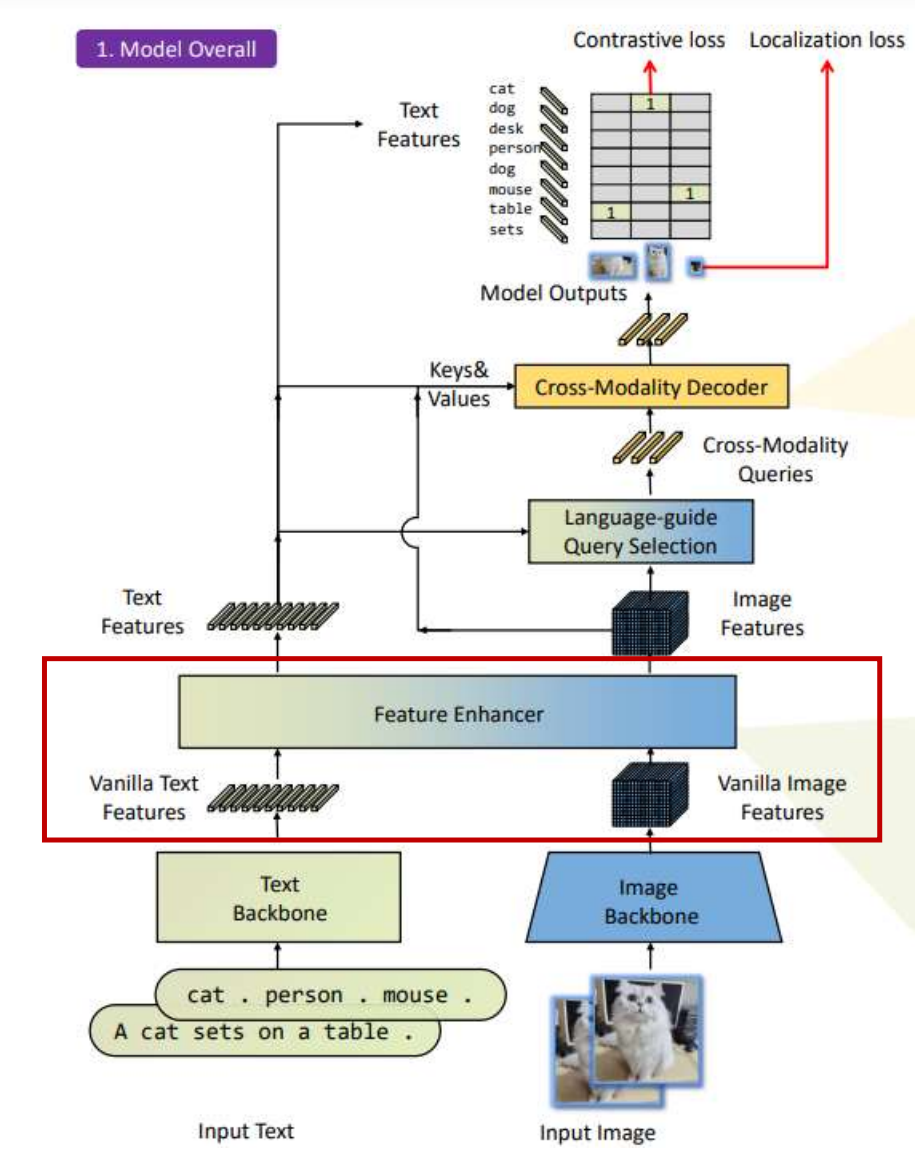
- Grounding DINO builds upon the DETR-like model DINO, transformer-based detector that leverages the strengths of the original DETR model.
- There are many other models built upon DETR
  - DAB-DETR
  - DN-DETR
- Mostly focused on closed-set object detection

Model Architecture

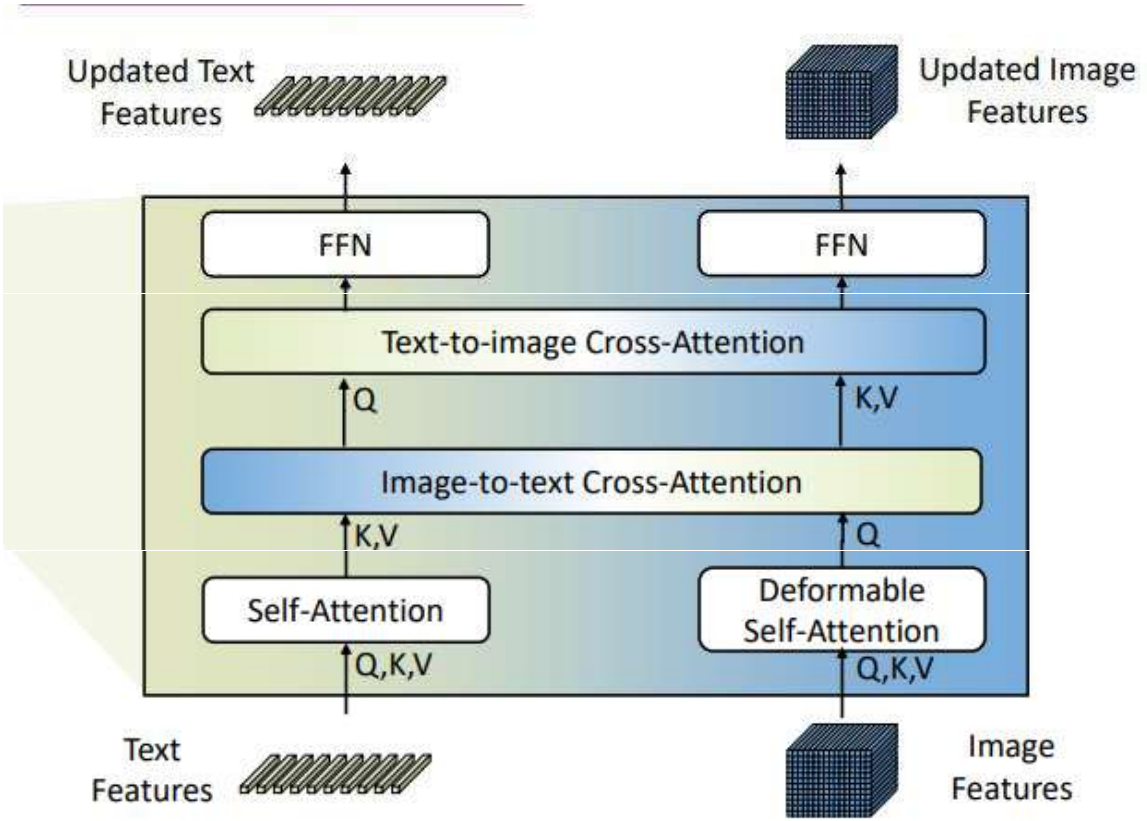
- Dual-encoder-single-decoder architecture
- Key Components :
  - Image Backbone
  - Text Backbone
  - Feature Enhancer
  - Language-Guided Query Selection
  - Cross-Modality Decoder



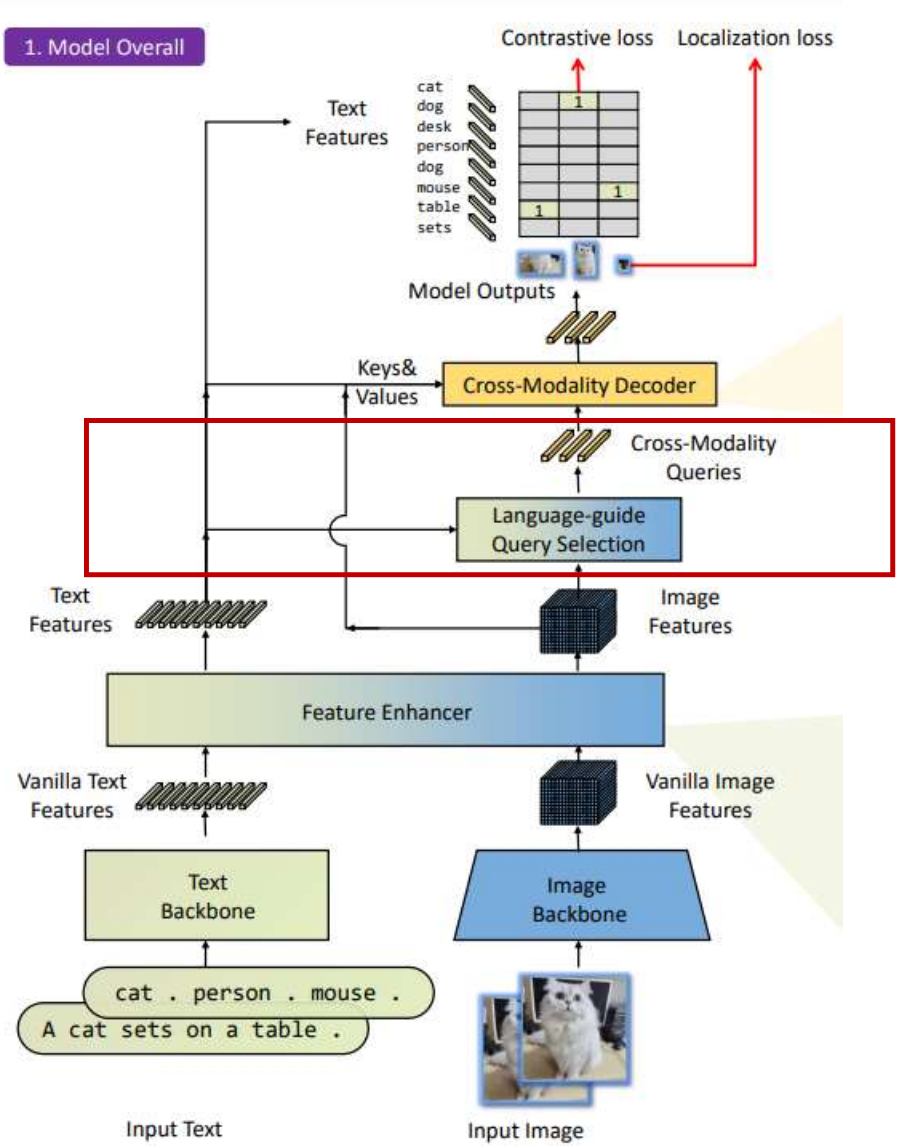
Feature Enhancer



Feature Enhancer



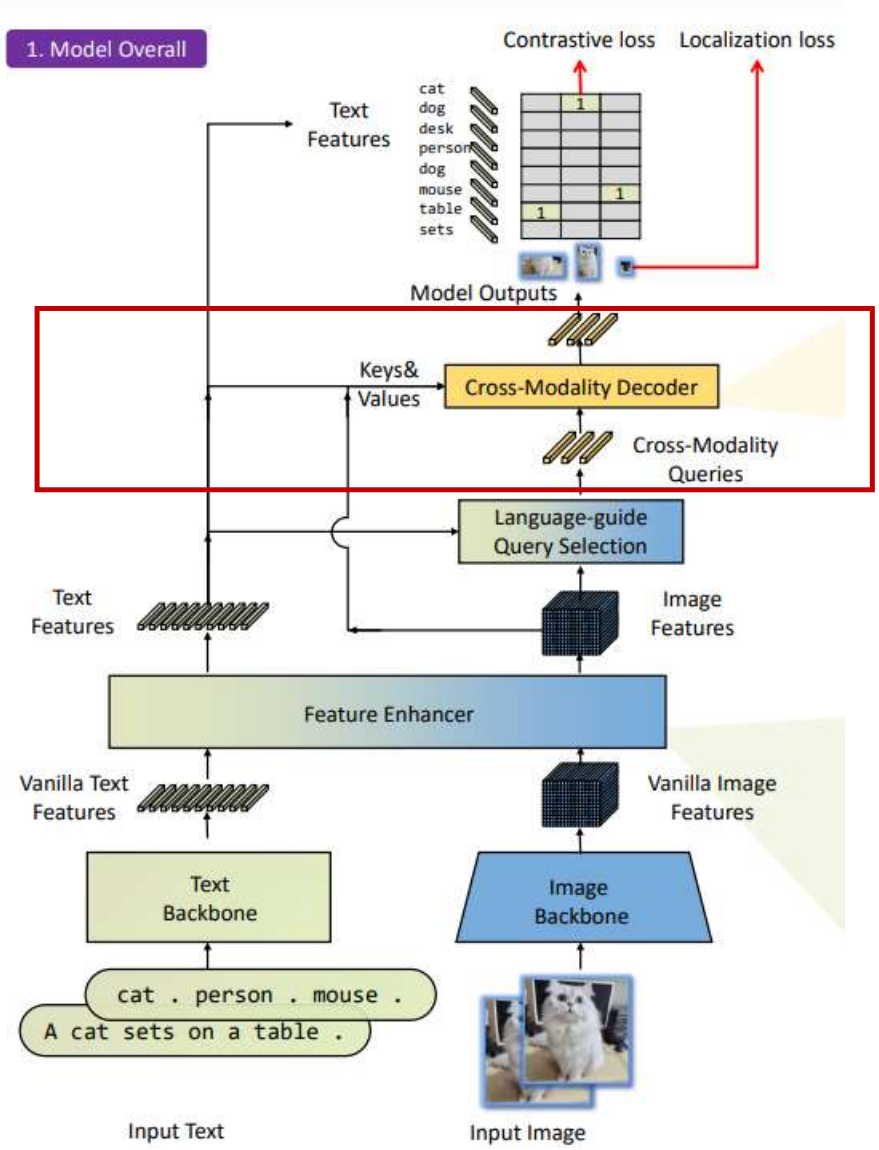
Language-Guided Query Selection



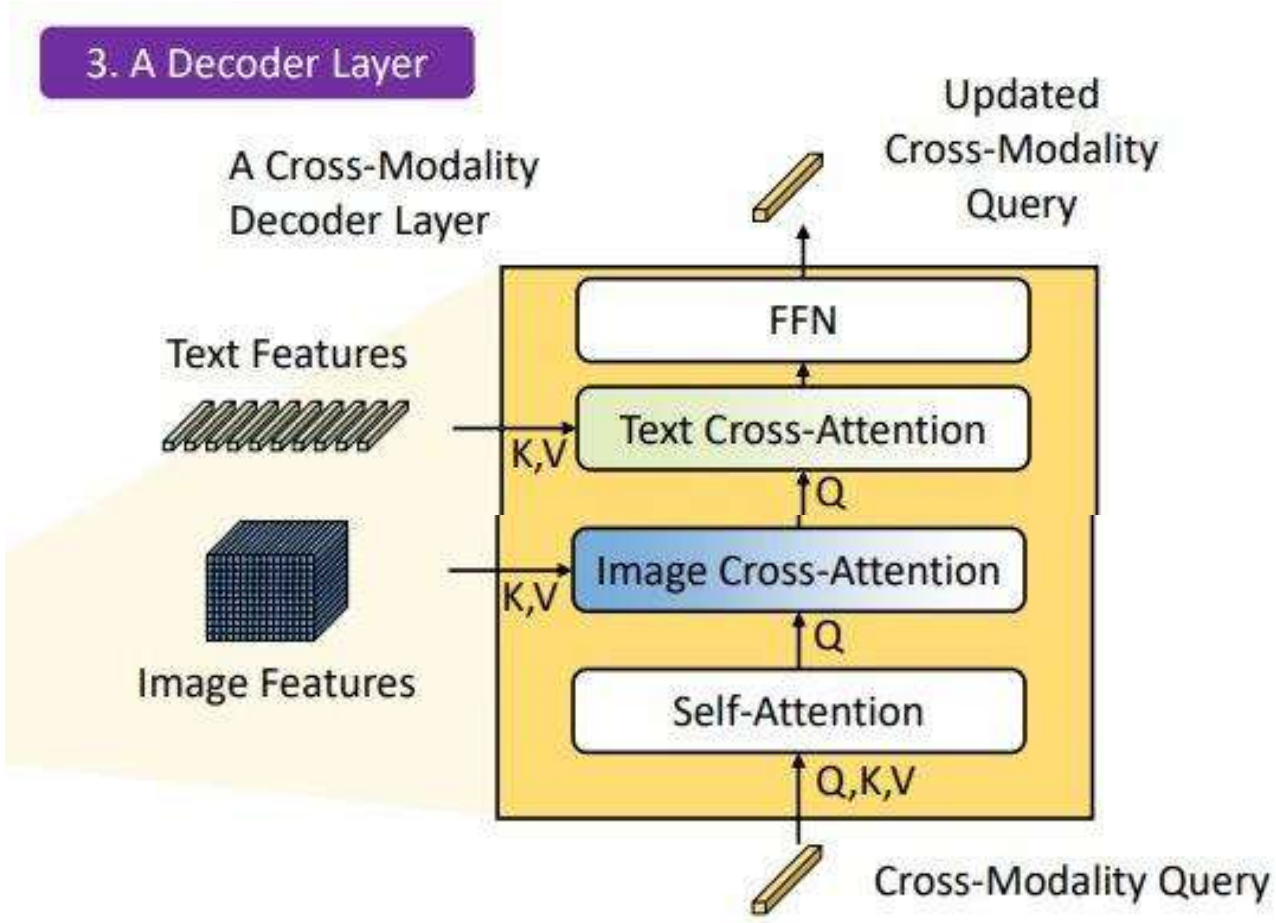
Language-Guided Query Selection

- Select image features that are more relevant to the text queries.
- Outputs cross modality queries for object detection

Cross-Modality Decoder

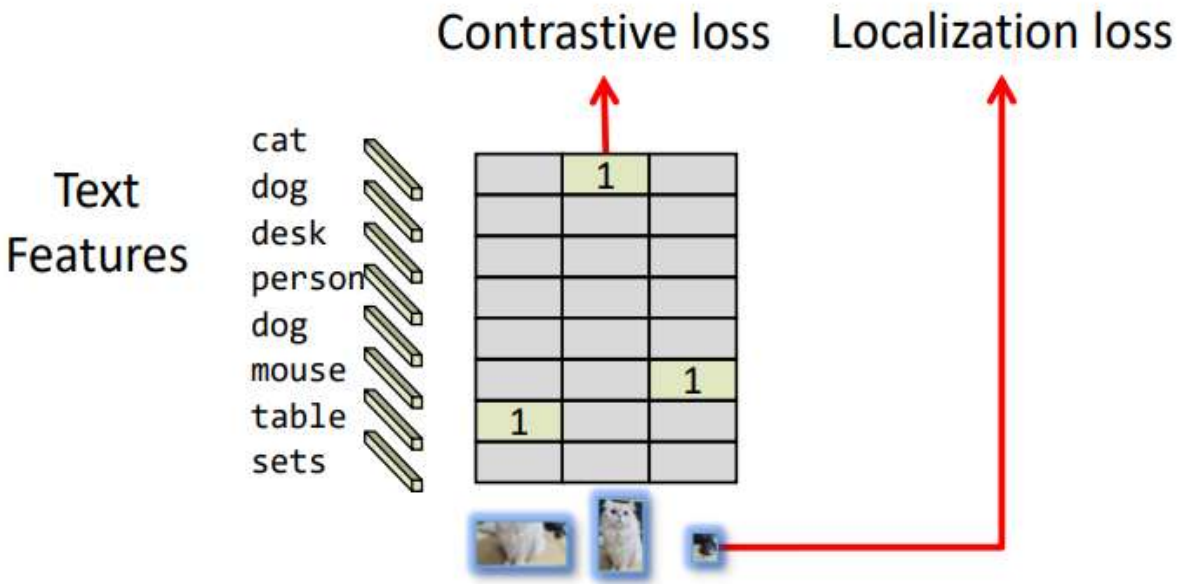


Cross-Modality Decoder



Loss Function

- Contrastive loss for classification
- L1 loss and GIOU for bounding box regression



Zero-Shot Transfer of Grounding DINO

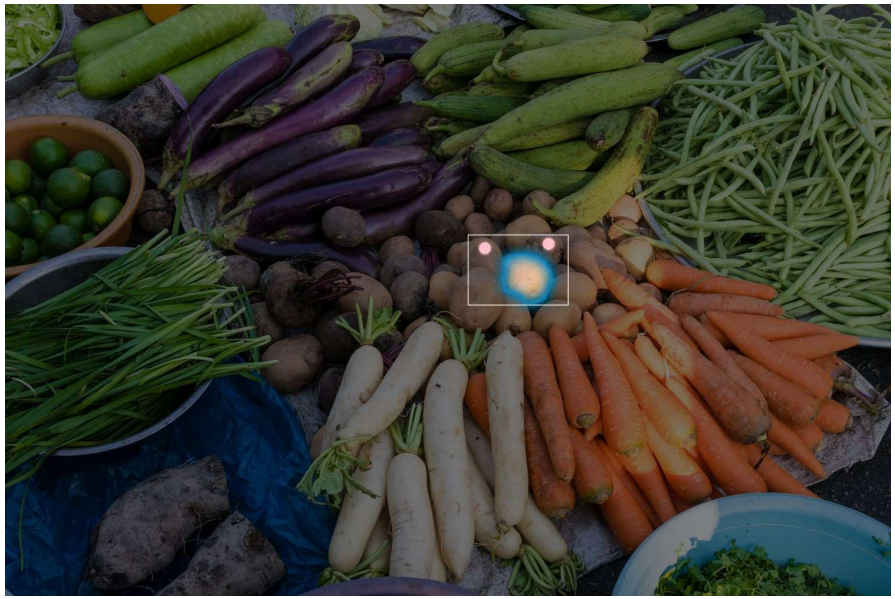
COCO Benchmark

Model	Backbone	Pre-Training Data	Zero-Shot 2017val	Fine-Tuning 2017val/test-dev
Faster R-CNN	RN50-FPN	-	-	40.2 / -
Faster R-CNN	RN101-FPN	-	-	42.0 / -
DyHead-T [5]	Swin-T	-	-	49.7 / -
DyHead-L [5]	Swin-L	-	-	58.4 / 58.7
DyHead-L [5]	Swin-L	O365,ImageNet21K	-	60.3 / 60.6
SoftTeacher [52]	Swin-L	O365,SS-COCO	-	60.7 / 61.3
DINO(Swin-L) [58]	Swin-L	O365	-	62.5 / -
DyHead-T† [5]	Swin-T	O365	43.6	53.3 / -
GLIP-T (B) [26]	Swin-T	O365	44.9	53.8 / -
GLIP-T (C) [26]	Swin-T	O365,GoldG	46.7	55.1 / -
GLIP-L [26]	Swin-L	FourODs,GoldG,Cap24M	49.8	60.8 / 61.0
DINO(Swin-T)† [58]	Swin-T	O365	46.2	56.9 / -
Grounding-DINO-T (Ours)	Swin-T	O365	46.7	56.9 / -
Grounding-DINO-T (Ours)	Swin-T	O365,GoldG	48.1	57.1 / -
Grounding-DINO-T (Ours)	Swin-T	O365,GoldG,Cap4M	48.4	57.2 / -
Grounding-DINO-L (Ours)	Swin-L	O365,OI [19],GoldG	**52.5**	**62.6** / **62.7** (**63.0** / **63.0**)*
Grounding-DINO-L (Ours)	Swin-L	O365,OI,GoldG,Cap4M,COCO,RefC	**60.7**	**62.6** / -

Segment Anything

Segment Anything introduces three innovations:

1. Promptable Segmentation Task: allows to find masks given:
    1. Points (for example points selected by the user by mouse click)
    2. Boxes (a rectangle defined by the user)
    3. Text prompt (for example "find all the dogs in this image")
    4. A combination of the above (for example a box and a background point)
2. Segment Anything Model
    1. A **fast** encoder-decoder model (takes 50ms in a web browser to generate a mask given a prompt)
    2. Ambiguity-aware: for example given a point, it may correspond to multiple masks and the model should return the three most likely (the **part**, the **subpart** and the **whole**).
3. Segment Anything Dataset (and its Segment Anything Engine)
    1. 1.1 billion segmentation masks collected with the Segment Anything Engine
    2. All the masks have been generated automatically! (no human supervision!)



Segment Anything Task

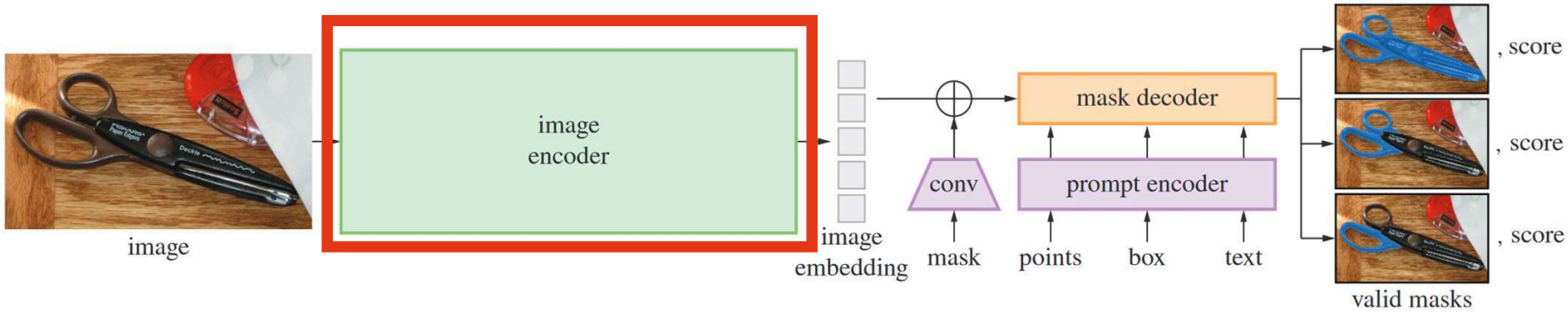
2. Segment Anything Task

We take inspiration from NLP, where the next token prediction task is used for foundation model pre-training *and* to solve diverse downstream tasks via prompt engineering [10]. To build a foundation model for segmentation, we aim to define a task with analogous capabilities.

Task. We start by translating the idea of a prompt from NLP to segmentation, where a prompt can be a set of foreground / background points, a rough box or mask, free-form text, or, in general, any information indicating what to segment in an image. The *promptable segmentation task*, then, is to return a *valid* segmentation mask given any *prompt*. The requirement of a "valid" mask simply means that even when a prompt is *ambiguous* and could refer to multiple objects (*e.g.*, recall the shirt *vs.* person example, and see Fig. 3), the output should be a reasonable mask for at least *one* of those objects. This requirement is similar to expecting a language model to output a coherent response to an ambiguous prompt. We choose this task because it leads to a natural pre-training algorithm *and* a general method for zero-shot transfer to downstream segmentation tasks via prompting.



Segment Anything Model



Segment Anything Model: Image Encoder

Image encoder. Motivated by scalability and powerful pre-training methods, we use an MAE [47] pre-trained Vision Transformer (ViT) [33] minimally adapted to process high resolution inputs [62]. The image encoder runs once per image and can be applied prior to prompting the model.

Vision Transformer

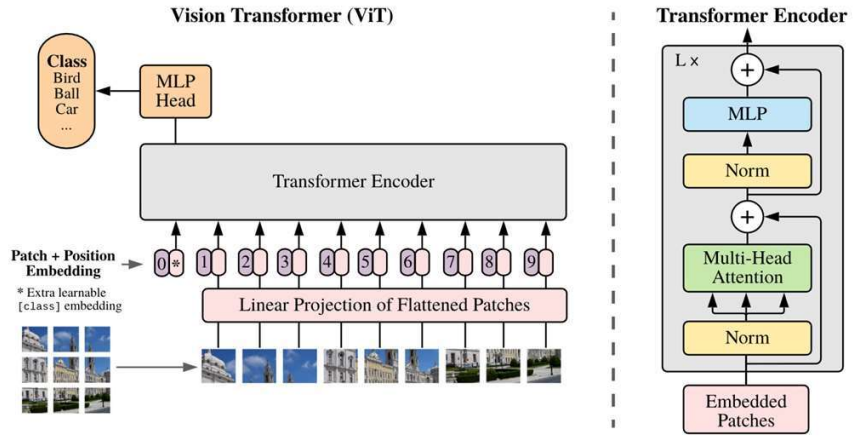


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al (2017).

AN IMAGE IS WORTH 16X16 WORDS:
TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy[*,†], **Lucas Beyer**[*], **Alexander Kolesnikov**[*], **Dirk Weissenborn**[*],
Xiaohua Zhai[*], **Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,**
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby[*,†]
[*]equal technical contribution, [†]equal advising
Google Research, Brain Team
{adosovitskiy, neilhoulsby}@google.com

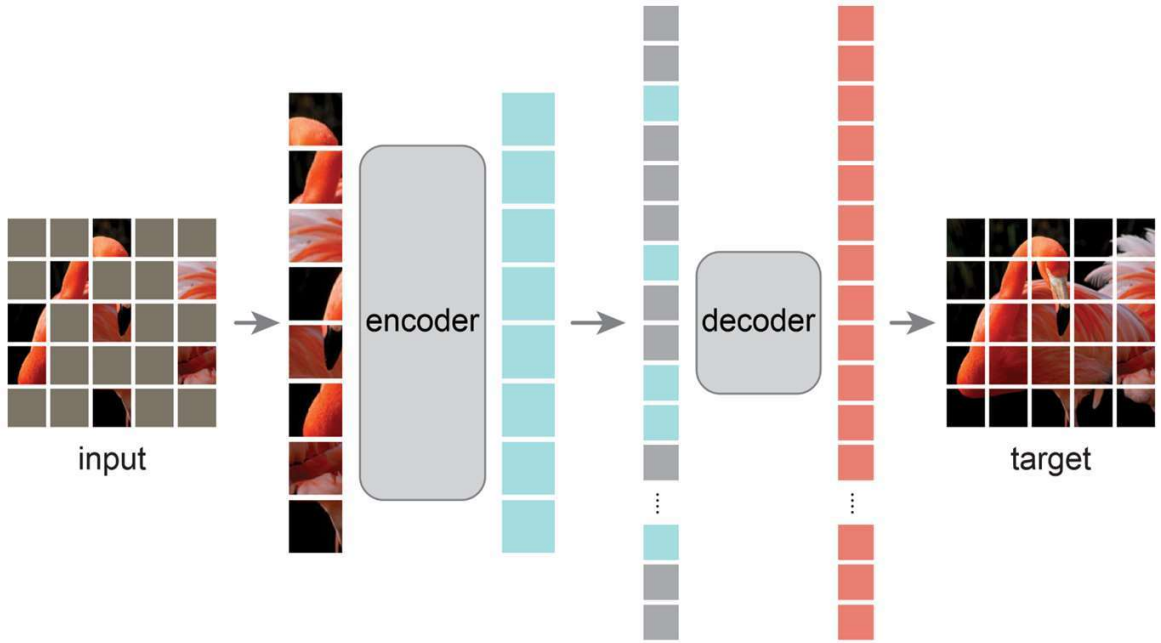
Masked Autoencoder Vision Transformer

Masked Autoencoders Are Scalable Vision Learners

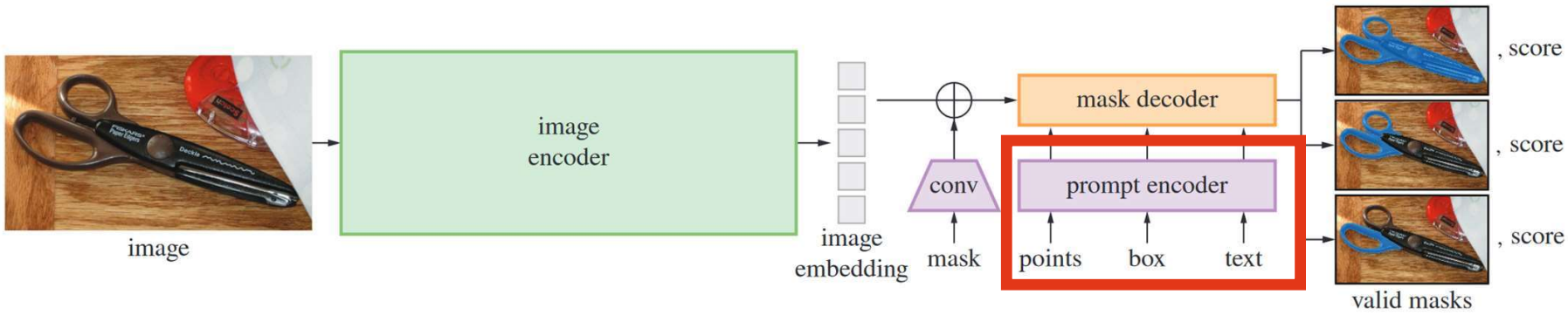
Kaiming He*,† Xinlei Chen* Saining Xie Yanghao Li Piotr Dollár Ross Girshick

*equal technical contribution †project lead

Facebook AI Research (FAIR)



Segment Anything Model



Segment Anything Model: Prompt Encoder

Prompt encoder. We consider two sets of prompts: *sparse* (points, boxes, text) and *dense* (masks). We represent points and boxes by positional encodings [95] summed with learned embeddings for each prompt type and free-form text with an off-the-shelf text encoder from CLIP [82]. Dense prompts (*i.e.*, masks) are embedded using convolutions and summed element-wise with the image embedding.

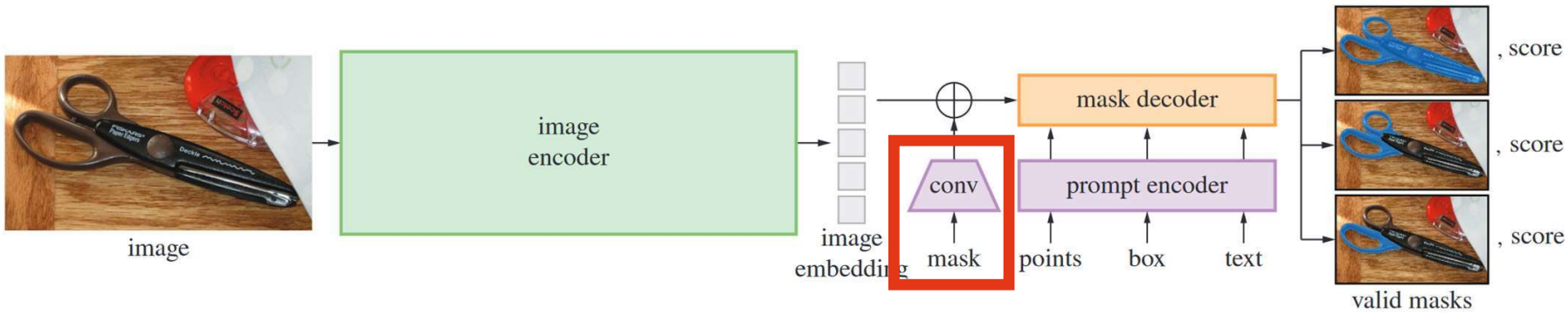
Segment Anything Model: Prompt Encoder

Prompt encoder. Sparse prompts are mapped to 256-dimensional vectorial embeddings as follows. A point is represented as the sum of a positional encoding [95] of the point's location and one of two learned embeddings that indicate if the point is either in the foreground or background. A box is represented by an embedding pair: (1) the positional encoding of its top-left corner summed with a learned embedding representing "top-left corner" and (2) the same structure but using a learned embedding indicating "bottom-right corner". Finally, to represent free-form text we use the text encoder from CLIP [82] (any text encoder is possible in general). We focus on geometric prompts for the remainder of this section and discuss text prompts in depth in §D.5.

```python
def _embed_points(
    self,
    points: torch.Tensor, # Indicates the coordinates of the points
    labels: torch.Tensor, # Indicate if the point is foreground or background
    pad: bool,
) -> torch.Tensor:
    """Embeds point prompts."""
    points = points + 0.5  # Shift to center of pixel
    if pad: # Add padding if needed (to keep the sequence length constant)
        padding_point = torch.zeros((points.shape[0], 1, 2), device=points.device)
        padding_label = -torch.ones((labels.shape[0], 1), device=labels.device) # The -1 label indicats padding
        points = torch.cat([points, padding_point], dim=1) # Append the padding point
        labels = torch.cat([labels, padding_label], dim=1) # Append the padding label
    point_embedding = self.pe_layer.forward_with_coords(points, self.input_image_size) # Obtain the positional encodings
    point_embedding[labels == -1] = 0.0 # zero out padding
    point_embedding[labels == -1] += self.not_a_point_embed.weight # Add special embedding to indicate padding
    point_embedding[labels == 0] += self.point_embeddings[0].weight # Add embedding for background points
    point_embedding[labels == 1] += self.point_embeddings[1].weight # Add embedding for foreground points
    return point_embedding

def _embed_boxes(self, boxes: torch.Tensor) -> torch.Tensor:
    """Embeds box prompts."""
    boxes = boxes + 0.5  # Shift to center of pixel
    coords = boxes.reshape(-1, 2, 2)
    corner_embedding = self.pe_layer.forward_with_coords(coords, self.input_image_size) # Obtain the positional encodings
    corner_embedding[:, 0, :] += self.point_embeddings[2].weight # Special embedding for top-left corner
    corner_embedding[:, 1, :] += self.point_embeddings[3].weight # Special embedding for bottom-right corner
    return corner_embedding
```

Segment Anything Model



Segment Anything Model: Mask Convolution

Dense prompts (*i.e.*, masks) have a spatial correspondence with the image. We input masks at a 4× lower resolution than the input image, then downscale an additional 4× using two 2×2, stride-2 convolutions with output channels 4 and 16, respectively. A final 1×1 convolution maps the channel dimension to 256. Each layer is separated by GELU activations [50] and layer normalization. The mask and image embedding are then added element-wise. If there is no mask prompt, a learned embedding representing "no mask" is added to each image embedding location.

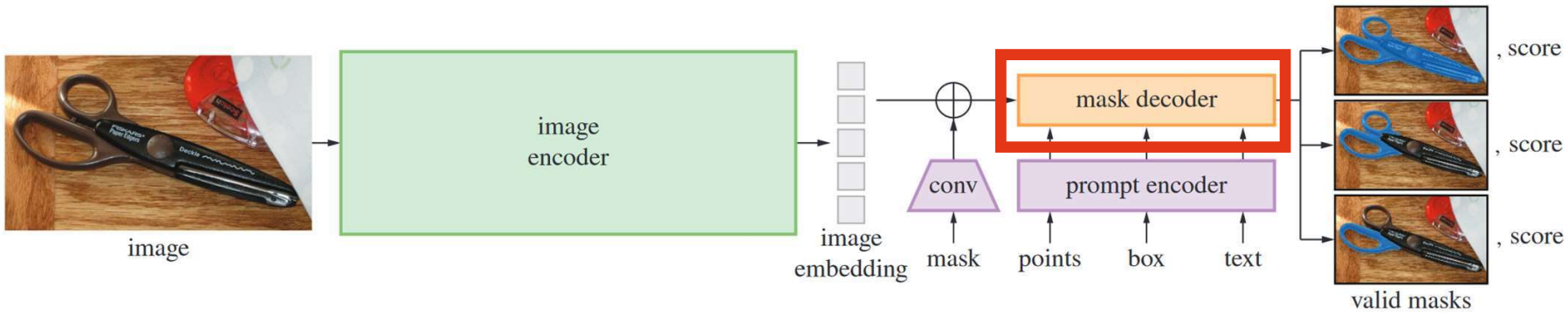
```python
if masks is not None:
    dense_embeddings = self._embed_masks(masks)
else: # If no mask is specified, use a special "no mask" embedding
    dense_embeddings = self.no_mask_embed.weight.reshape(1, -1, 1, 1).expand(
        bs, -1, self.image_embedding_size[0], self.image_embedding_size[1]
    )
```

```python
def _embed_masks(self, masks: torch.Tensor) -> torch.Tensor:
    """Embeds mask inputs."""
    mask_embedding = self.mask_downscaling(masks)
    return mask_embedding
```

```python
self.mask_downscaling = nn.Sequential(
    nn.Conv2d(1, mask_in_chans // 4, kernel_size=2, stride=2),
    LayerNorm2d(mask_in_chans // 4),
    activation(),
    nn.Conv2d(mask_in_chans // 4, mask_in_chans, kernel_size=2, stride=2),
    LayerNorm2d(mask_in_chans),
    activation(),
    nn.Conv2d(mask_in_chans, embed_dim, kernel_size=1),
)
self.no_mask_embed = nn.Embedding(1, embed_dim)
```

```python
# Expand per-image data in batch direction to be per-mask
src = torch.repeat_interleave(image_embeddings, tokens.shape[0], dim=0)
src = src + dense_prompt_embeddings # Add mask embeddings to the image
pos_src = torch.repeat_interleave(image_pe, tokens.shape[0], dim=0)
b, c, h, w = src.shape
```

Segment Anything Model

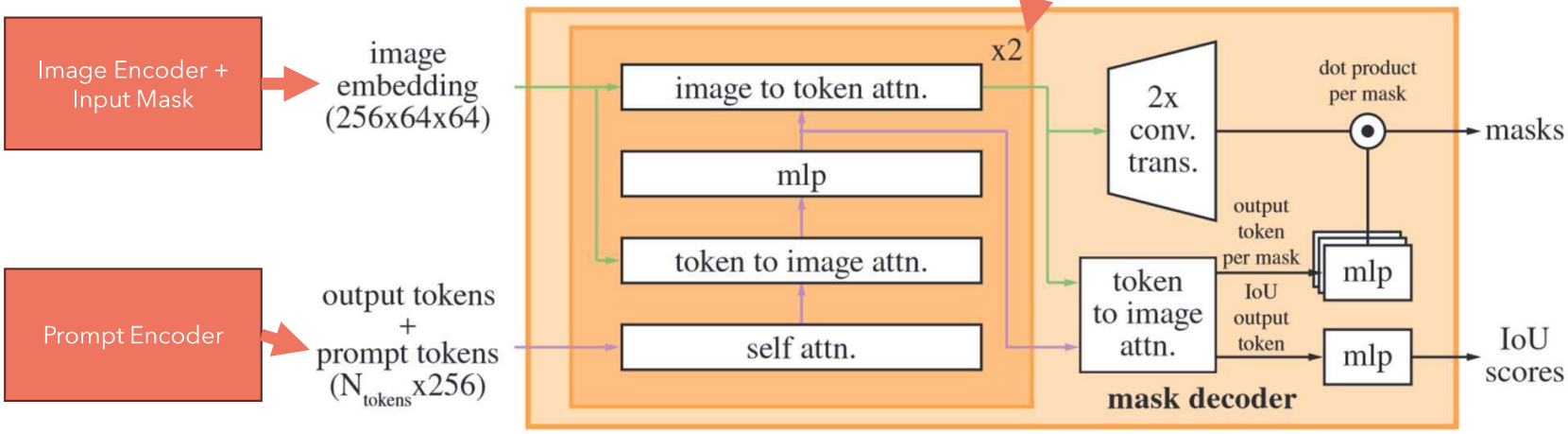


Segment Anything Model: Mask Decoder

We have two layers, one on top of the other (output of the first is the input of the second).

```
# Prepare queries
queries = point_embedding
keys = image_embedding

# Apply transformer blocks and final layernorm
for layer in self.layers:
    queries, keys = layer(
        queries=queries,
        keys=keys,
        query_pe=point_embedding,
        key_pe=image_pe,
    )
```



IoU (Intersection over Union)

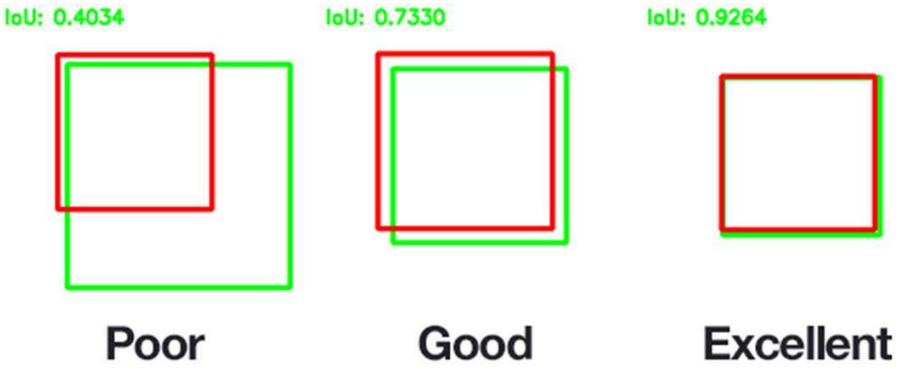
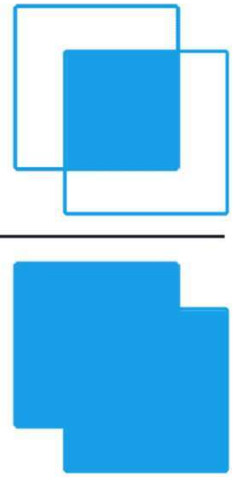


Image from Wikipedia

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Segment Anything Model: Loss

Losses. We supervise mask prediction with a linear combination of focal loss [65] and dice loss [73] in a 20:1 ratio of focal loss to dice loss, following [20, 14]. Unlike [20, 14], we observe that auxiliary deep supervision after each decoder layer is unhelpful. The IoU prediction head is trained with mean-square-error loss between the IoU prediction and the predicted mask's IoU with the ground truth mask. It is added to the mask loss with a constant scaling factor of 1.0.

Segment Anything Model: Focal loss

Focal Loss: very used in object detection and it is like Cross Entropy but adjusted for class imbalance.

That is, it allows the model to put more focus on hard examples and less on easy ones.

Why is there class imbalance? Because most of our pixels will be non-mask and only a small percentage will be part of the mask.

3.2. Focal Loss Definition

As our experiments will show, the large class imbalance encountered during training of dense detectors overwhelms the cross entropy loss. Easily classified negatives comprise the majority of the loss and dominate the gradient. While α balances the importance of positive/negative examples, it does not differentiate between easy/hard examples. Instead, we propose to reshape the loss function to down-weight easy examples and thus focus training on hard negatives.

More formally, we propose to add a modulating factor $(1 - p_t)^\gamma$ to the cross entropy loss, with tunable *focusing* parameter $\gamma \geq 0$. We define the focal loss as:

$$\mathrm{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t). \tag{4}$$

Focal Loss for Dense Object Detection

Tsung-Yi Lin Priya Goyal Ross Girshick Kaiming He Piotr Dollár
Facebook AI Research (FAIR)

Segment Anything Model: Dice loss

The Dice score, also known as the Sørensen–Dice coefficient, is a measure of the similarity between two sets of data. In the context of image segmentation, the Dice score can be used to evaluate the similarity between a predicted segmentation mask and the ground truth segmentation mask. The Dice score ranges from 0, indicating no overlap, to 1, indicating perfect overlap. It is equal to the F1 score

$$\text{Dice} = \frac{2\ X\ Area\ of\ overlap}{Total\ area} = \frac{2 \times \text{Prediction} \cap \text{Ground truth}}{\text{Prediction} + \text{Ground truth}}$$

https://medium.com/mlearning-ai/understanding-evaluation-metrics-in-medical-image-segmentation-d289a373a3f

$$Dice\ \boldsymbol{Loss} = 1\ - Dice\ score$$

3 Dice loss layer

The network predictions, which consist of two volumes having the same resolution as the original input data, are processed through a soft-max layer which outputs the probability of each voxel to belong to foreground and to background. In medical volumes such as the ones we are processing in this work, it is not uncommon that the anatomy of interest occupies only a very small region of the scan. This often causes the learning process to get trapped in local minima of the loss function yielding a network whose predictions are strongly biased towards background. As a result the foreground region is often missing or only partially detected. Several previous approaches resorted to loss functions based on sample re-weighting where foreground regions are given more importance than background ones during learning. In this work we propose a novel objective function based on dice coefficient, which is a quantity ranging between 0 and 1 which we aim to maximise. The dice coefficient D between two binary volumes can be written as

$$D = \frac{2\sum_i^N p_i g_i}{\sum_i^N p_i^2 + \sum_i^N g_i^2}$$

V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation

Fausto Milletari[1], Nassir Navab[1,2], Seyed-Ahmad Ahmadi[3]

PAPER WALKTHROUGH · CVPR 2024

Depth Anything

A foundation model for monocular depth estimation, built by unleashing large-scale unlabeled data

Lihe Yang et al., 2024 · Based on Sascha Kirch's walkthrough in Towards Data Science

THE PROBLEM

Depth from a single camera

To move through 3D space, a system must know where things are and how far away. Collision avoidance, drivable space, AR placement, robot grasping all depend on it.

Depth sensors

LiDAR, stereo and ToF are accurate, but expensive, bulky and complex to set up.

A single camera

Cheap, light, small, and already on almost every system.

The catch: it's ill-posed

Is the object small or far away? Is the surface concave or convex? 2D alone can't say.

BEFORE DEPTH ANYTHING

Many methods, each tied to one setting

Regression

Predict a depth value per pixel directly.

Classification

Discretize depth into bins and predict the bin.

Generative

GANs, VAEs and diffusion models that generate the depth map.

The limitation

Usually application-specific: indoor vs outdoor, relative vs metric, sparse vs dense.

The opportunity

Foundation models generalize zero-shot, but need web-scale data. Images and text are online; depth labels are not.

IN A NUTSHELL

An encoder–decoder trained in up to three stages

Image in, dense depth map out. Trained on a mix of labeled and unlabeled images.

1

Train a teacher

On 1.5M labeled images, with a pre-trained DINOv2 encoder.

2

Pretrain the student

The teacher pseudo-labels 62M unlabeled images. This student is Depth Anything.

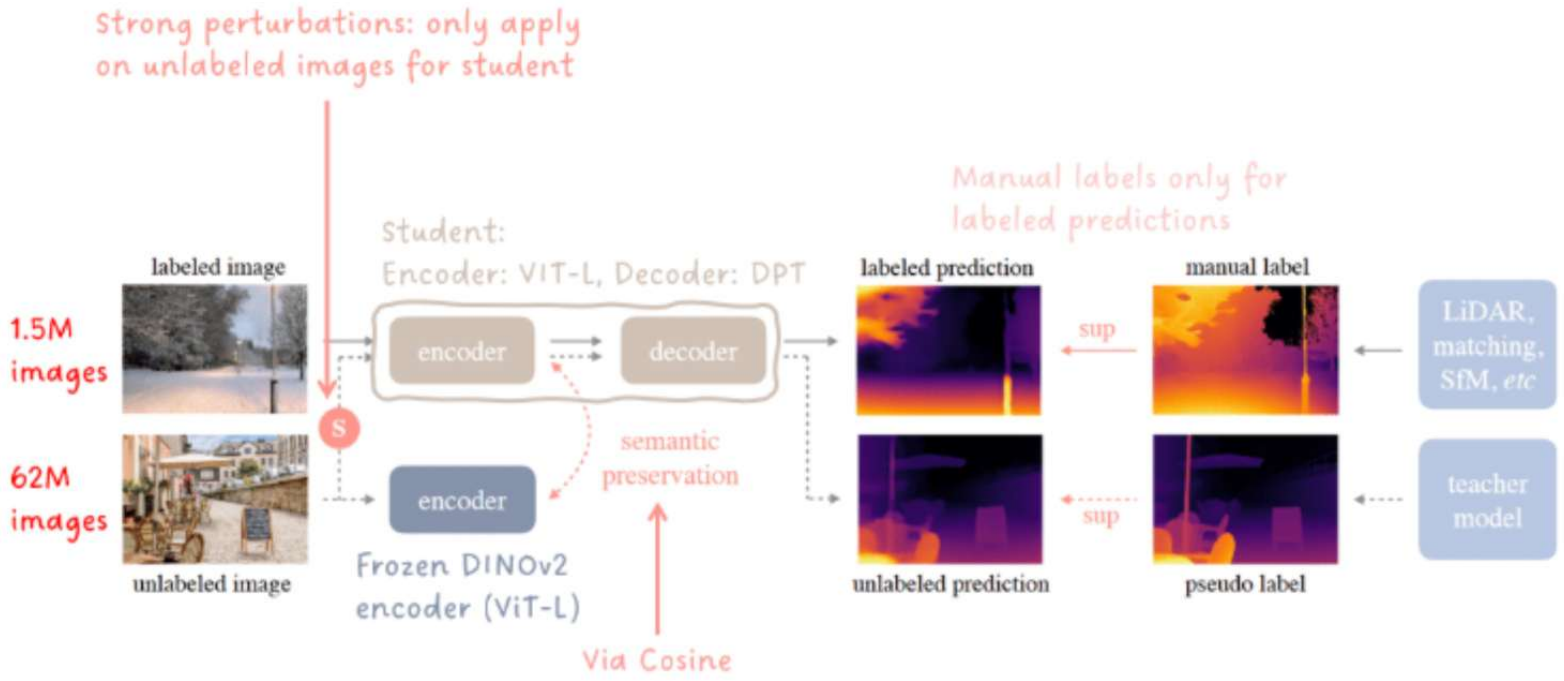
3

Fine-tune (optional)

Adapt to a specific task or dataset, e.g. metric depth or segmentation.

Two paths into one student

THE PIPELINE



Top: labeled path

Supervised by real labels from LiDAR, matching or SfM.

Bottom: unlabeled path

Strongly perturbed (S), supervised by teacher pseudo labels.

Dashed: semantic preservation

Student features kept close to frozen DINOv2.

Figure: Yang et al., 2024

THE DATA ENGINE

A teacher turns cheap images into labels

1.5M
labeled images (dense depth maps)

62M
unlabeled images, indoor and outdoor

Teacher

DINOv2 encoder, trained on labeled data only. Generates pseudo labels in place of slow SfM or stereo pipelines.

Student

Same architecture, but not the teacher's weights: DINOv2 encoder, randomly initialized decoder. Trained on both sets.

TRICK 1 · STRONG PERTURBATIONS

Make the student's job harder

Adding unlabeled data alone did **not** beat the labeled-only baseline. The fix: perturb the student's input, while the teacher sees the clean image.

Color jitter
Shift brightness, contrast and hue.

Noise
Corrupt pixels with image noise.

Spatial distortion
Warp the geometry of the image.

CutMix (50%)
Paste two images together with a mask.

Same intuition as DINO and BYOL: a car is still a car, blurred or mirrored.

TRICK 2 · FEATURE ALIGNMENT

Borrow semantics from a frozen DINOv2

Better scene understanding helps resolve depth ambiguity. The question was how to inject it.

TRIED, FAILED

Auxiliary segmentation

Shared encoder with separate depth and segmentation decoders, on foundation-model labels.

WHAT WORKED

Align encoder features

Keep student features close to frozen DINOv2, measured by cosine similarity.

THE CAVEAT

Margin α = 0.15

DINOv2 sees a car's front and rear as alike; depth doesn't. So don't enforce similarity fully.

TRAINING OBJECTIVE

The student averages three losses

$$L = (L_l + L_{feat} + L_u) / 3$$

Term	Compares	Form
L_l · labeled	Student prediction vs ground truth	Mean absolute error
L_{feat} · alignment	Student encoder vs frozen DINOv2	1 − mean cosine similarity, with margin α
L_u · unlabeled	Student on perturbed or CutMix image vs teacher pseudo label	MAE, computed separately for each image region of mask M

Three losses

Loss for labeled images

$$\mathcal{L}_l = \frac{1}{HW} \sum_{i=1}^{HW} \rho(d_i^*, d_i)$$

Mean absolute error function

Predicted depth

Real depth

Average over all image pixels.

Multiply with corresponding mask to ensure the loss only contributes to the correct region of predictions

Loss for unlabeled images

$$\mathcal{L}_u = \frac{\sum M}{HW} \mathcal{L}_u^M + \frac{\sum (1-M)}{HW} \mathcal{L}_u^{1-M}$$

Average over all image

Individual loss for the valid regions in the image

1- to construct a loss from a similarity.

Feature alignment loss

$$\mathcal{L}_{feat} = 1 - \frac{1}{HW} \sum_{i=1}^{HW} \cos(f_i, f_i')$$

Cosine similarity

Feature embeddings student

Feature embeddings DINOv2

Average over all image pixels.

Corresponding mask for the respective loss term

Mean absolute error function

Predicted depth of individual unlabeled images using teacher model

Loss for unlabeled images for individual masks

$$\mathcal{L}_u^M = \rho(S(u_{ab}) \odot M, T(u_a) \odot M),$$

$$\mathcal{L}_u^{1-M} = \rho(S(u_{ab}) \odot (1-M), T(u_b) \odot (1-M))$$

Predicted depth of CutMix image using student model